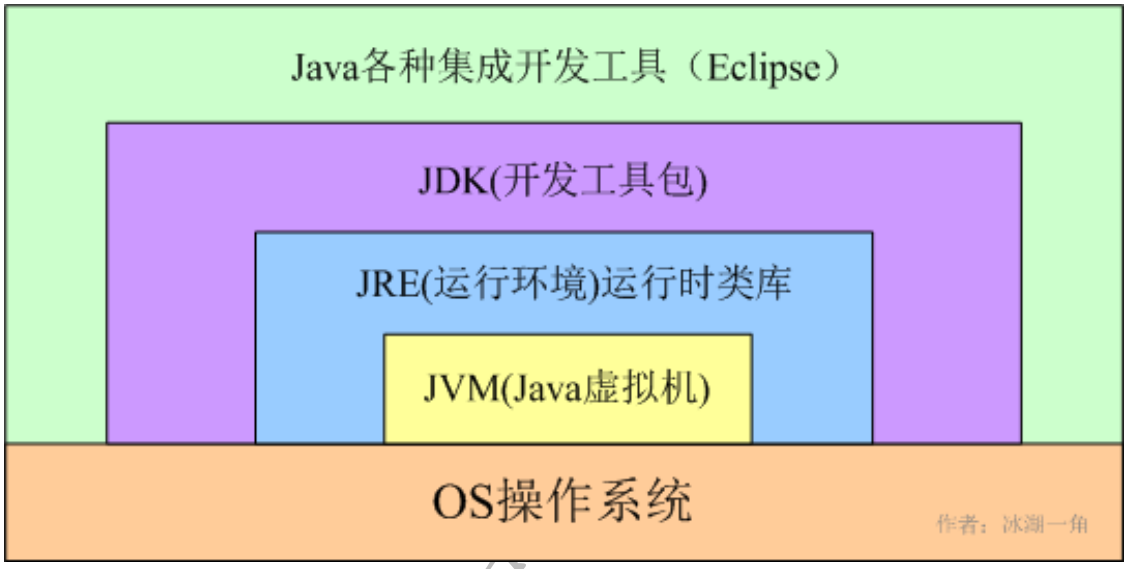
1 JVM 前奏篇

1.1 JDK、JRE、JVM

Java 程序是运行在 JVM(Java 虚拟机)上的，在开发程序之前都要配置 Java 开发环境，其中首先要做的就是 JDK 的安装和配置，那么 JDK、JVM、JRE 到底有何联系和区别呢？想必并不是每一个程序员都能说得清楚的，接下来将带你了解它们之间的关系。



1.1.1 JDK

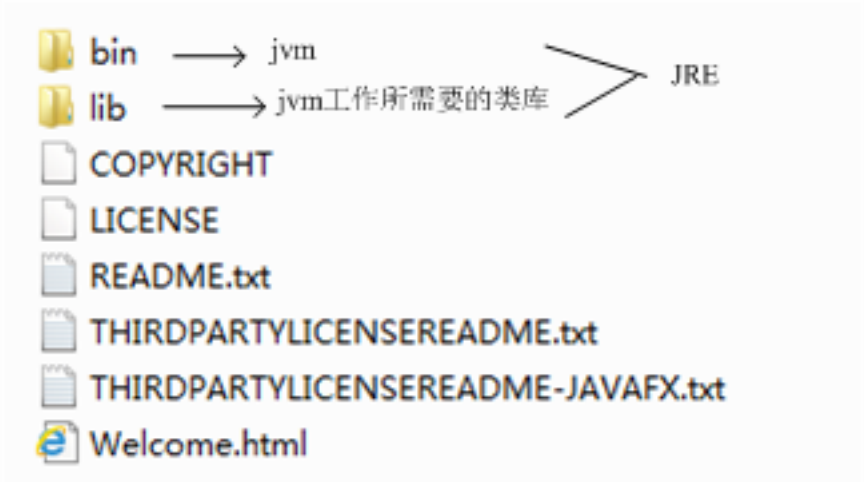
JDK(Java SE Development Kit)，Java 标准开发包，它提供了编译、运行 Java 程序所需的各种工具和资源，包括 Java 编译器、Java 运行时环境，以及常用的 Java 类库等。

下图是 JDK 的安装目录：

1.1.2 JRE

JRE(Java Runtime Environment) 、Java 运行环境，用于解释执行 Java 的字节码文件。普通用户而只需要安装 JRE（Java Runtime Environment）来运行 Java 程序。而程序开发者必须安装 JDK 来编译、调试程序。

下图是 JRE 的安装目录：里面有两个文件夹 bin 和 lib，在这里可以认为 bin 里的就是 jvm，lib 中则是 jvm 工作所需要的类库，而 jvm 和 lib 和起来就称为 jre。



1.1.3 JVM

JVM(Java Virtual Machine)，Java 虚拟机，是 JRE 的一部分。它是整个 java 实现跨平台的最核心的部分，负责解释执行字节码文件，是可运行 java 字节码文件的虚拟计算机。所有平台的上的 JVM 向编译器提供相同的接口，而编译器只需要面向虚拟机，生成虚拟机能识别的代码，然后由虚拟机来解释执行。

当使用 Java 编译器编译 Java 程序时，生成的是与平台无关的字节码，这些字节码只面向 JVM。不同平台的 JVM 都是不同的，但它们都提供了相同的接口。JVM 是 Java 程序跨平台的关键部分，只要为不同平台实现了相应的虚拟机，编译后的 Java 字节码就可以在该平台上运行。

1.1.4 区别与联系

1. JDK 用于开发，JRE 用于运行 java 程序；如果只是运行 Java 程序，可以只安装 JRE，无需安装 JDK。
2. JDk 包含 JRE，JDK 和 JRE 中都包含 JVM。
3. JVM 是 java 编程语言的核心并且具有平台独立性。

官网链接：https://docs.oracle.com/javase/8/docs/index.html

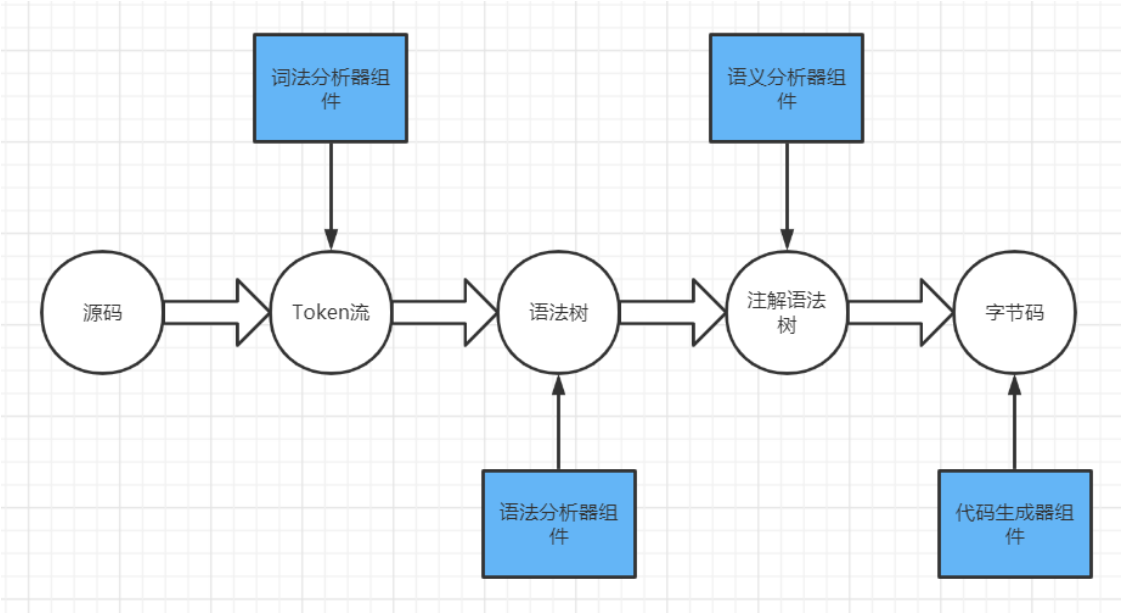
1.2 java 变 class

也就是我们说的编译过程，由 java 编译器把 java 文件变成 class 的字节码文件

javac ClassB.java

javap -verbose ClassB.class

咕泡出品 必属精品



词法分析器：读取源代码，一个字节一个字节的读取出来，找出这个词法中 if,else,while 等

结论：就是分析源码哪些是标点符号，哪些是动词，哪些是名词，并转化成 Token 流形式

语法分析器：分析 Token 流，检查是否符合 Java 语言规范

结论：检查 Token 流是否符合 JAVA 语言规范，有没有语法错误。

例如：if 后面是否跟着 boolean 表达式

语义分析器：主要工作就是将比较复杂的语法语义转化成简单的语法，

就如难懂的文言文转化为大家都懂的百话文，或者是注释一下一些不懂的成语。

结论：就是将复杂的语法转化为简单语法。例如：foreach 转化为 for 循环的过程

代码生成器：将注解语法树生转化成字节码的过程

结论：代码生成器的结果就是生成符合 java 虚拟机规范的字节码

https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-4.html#jvms-4.1

ClassFile 结构

```
ClassFile {
    u4              magic;    //魔法数，Class 文件的标识。值是固定的，为0xCA
FEBABE
    u2              minor_version;    //小版本
```

```
    u2              major_version;  //主版本号 34，52 对应java 8 ,也就是我们jdk8 生成的class 版本
    u2              constant_pool_count;   //38  常量池计算器
    cp_info         constant_pool[constant_pool_count-1];   //常量池
    u2              access_flags;  //访问标志  标志了类或者接口的访问信息，比如是类还是接口还是注解、枚举，是否是abstract,如果是类，是否被声明成final等等
    u2              this_class;  //类索引
    u2              super_class; //父类索引
    u2              interfaces_count;   //接口计数器  ，这也是之前大家在讨论动态代理的时候最多是65535 个接口的时候讨论过的
    u2              interfaces[interfaces_count];
    u2              fields_count;   //字段个数
    field_info      fields[fields_count];  //字段集合
    u2              methods_count;   //方法计数器
    method_info     methods[methods_count];  //方法集合
    u2              attributes_count;  //附加属性计数器
    attribute_info attributes[attributes_count];   //附加属性集合
}
```

我们的 class 文件发现有 2 种数据类型

u 几 u 几，这种就是无符号数，属于基本的数据类型，用来描述数字、索引引用，数量值等，以 u1、u2、u4、u8 来分别代表 1 个字节、2 个字节、4 个字节和 8 个字节

_info 结尾的，这种数据类型叫做表，都是以 _info 结尾

magic

魔数，区分文件类型的一种标志，一般都是文件的前几个字节来表示，比如 0XCAFE BABE 表示 class 文件，因为我们 java 的图标是杯 cafe，因为为了蹭当时的热度，当时咖啡最盛产的地方就是爪哇岛，所以名字叫 java,图标就是 cafe，然后 java bean 就是咖啡豆

minor_version major_version

次要和主要版本号，一起确定 class 文件格式的版本

https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-4.html#jvms-4.7 有版本说明！！

所以我们的这个 class 的版本是 52，所以是 jdk1.8 编译

constant_pool_count & constant_pool[constant_pool_count-1]

哈哈出品 必属精品

数量，会比实际多 1

我们写代码时都是从 0 开始的，但是这里的常量池却是从 1 开始，因为它把第 0 项常量空出来了。这是为了在于满足后面某些指向常量池的索引值的数据在特定情况下需要表达“不引用任何一个常量池项目”的含义，这种情况可用索引值 0 来表示

我们可以通过命令查看常量池信息

```
javap -verbose ClassB.class
```

类 型 上面的tag对应这里的标志	标 志	描 述
CONSTANT_utf8_info	1	UTF-8编码的字符串
CONSTANT_Integer_info	3	整形字面量
CONSTANT_Float_info	4	浮点型字面量
CONSTANT_Long_info	5	长整型字面量
CONSTANT_Double_info	6	双精度浮点型字面量
CONSTANT_Class_info	7	类或接口的符号引用
CONSTANT_String_info	8	字符串类型字面量
CONSTANT_Fieldref_info	9	字段的符号引用
CONSTANT_Methodref_info	10	类中方法的符号引用
CONSTANT_InterfaceMethodref_info	11	接口中方法的符号引用
CONSTANT_NameAndType_info	12	字段或方法的符号引用
CONSTANT_MethodHandle_info	15	表示方法句柄
CONSTANT_MothodType_info	16	标志方法类型
CONSTANT_InvokeDynamic_info	18	表示一个动态方法调用点

Class文件结构中常量池中11种数据类型的结构总表			
常量	项目	类型	描述
CONSTANT_Utf8_info	tag	U1	值为1
	length	U2	UTF-8编码的字符串长度
	bytes	U1	长度为length的UTF-8编码的字符串
CONSTANT_Integer_info	tag	U1	值为3
	bytes	U4	按照高位在前存储的int值
CONSTANT_Float_info	tag	U1	值为4
	bytes	U4	按照高位在前存储的float值
CONSTANT_Long_info	tag	U1	值为5
	bytes	U8	按照高位在前存储的long值
CONSTANT_Double_info	tag	U1	值为6
	bytes	U8	按照高位在前存储的double值
CONSTANT_Class_info	tag	U1	值为7
	index	U2	指向全限定名常量项的索引
CONSTANT_String_info	tag	U1	值为8
	index	U2	指向字符串字面量的索引
CONSTANT_Fieldref_info	tag	U1	值为9
	index	U2	指向声明字段的类或者接口描述符CONSTANT_Class_info的索引项
	index	U2	指向字段描述符CONSTANT_NameAndType_info的索引项
CONSTANT_Methodref_info	tag	U1	值为10
	index	U2	指向声明方法的类描述符CONSTANT_Class_info的索引项
	index	U2	指向名称及类型描述符CONSTANT_NameAndType_info的索引项
CONSTANT_InterfaceMethodref_info	tag	U1	值为11
	index	U2	指向声明方法的接口描述符CONSTANT_Class_info的索引项
	index	U2	指向名称及类型描述符CONSTANT_NameAndType_info的索引项
CONSTANT_NameAndType_info	tag	U1	值为12
	index	U2	指向该字段或方法名称常量项的索引
	index	U2	指向该字段或方法描述符常量项的索引

常量池各个表结构

符号引用：类和接口的全限定名、字段的名称和描述符、方法的名称和描述符

访问标志：

```
00000000 00000001：public
00000000 00010000：final
00000000 00100000：super
00000010 00000000：interface
00000100 00000000：abstract
00010000 00000000：synthetic
00100000 00000000：annotation
01000000 00000000：enum
```

1.3 class 类的生命周期

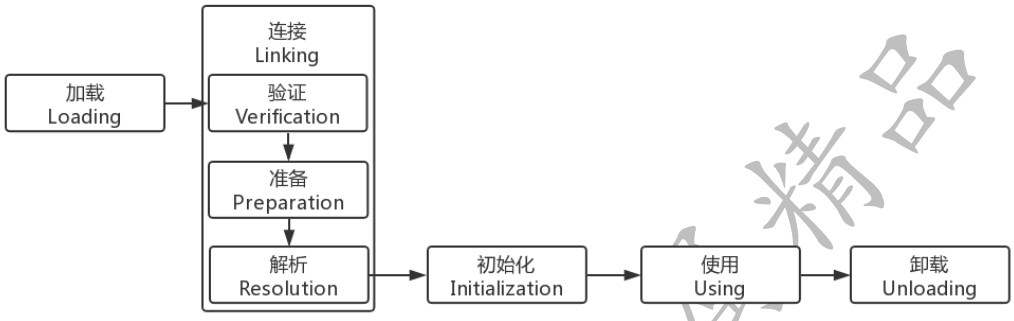
也就是我们经常讲的类加载！！！把 class 类加载到 JVM

虚拟机把描述类的数据从 Class 文件加载到内存，并对数据进行校验、转换解析和初始化，最终形成可以被虚拟机直接使用的 Java 类型，这就是虚拟机的类加载机制。

在 Java 语言里面，类型的加载、连接和初始化过程都是在程序运行期间完成的

1.3.1 类加载的过程

类的整个生命周期如下图：



为支持运行时绑定，解析过程在某些情况下可在初始化之后再开始，除解析过程外的其他加载过程必须按照如图顺序开始。

1.3.2 加载（load）

1. 通过全限定类名来获取定义此类的二进制字节流。（先找到类文件所在的位置：磁盘的全路径）
2. 将这个字节流所代表的静态存储结构转化为方法区的运行时数据结构。（类文件的信息交给 JVM）
3. 在内存中生成一个代表这个类的 java.lang.Class 对象，作为方法区这个类的各种数据的访问入口。（类文件所对应的对象 Class ---->JVM）

1.3.2.1 引导类加载器（Bootstrap ClassLoader ）(启动类加载器)

加载$JAVA_HOME 中 jre/lib/rt.jar 里所有的 class 或 Xbootclassoath 选项指定的 jar 包! 这些类是 Java 运行的基础类，BootstrapClassLoader 比较特殊，它不继承 ClassLoader，而是由 JVM 内部实现；

C++实现，并非 java 代码实现，所以在 JVM 虚拟机规范中的支持两种类型的类加载器，分别为引导类加载器（Bootstrap ClassLoader）和自定义类加载器（User-

Defin ed ClassLoader），什么拓展类，应用程序类、自动类都可以称为自定义类加载器

https://docs.oracle.com/javase/specs/jvms/se8/html/jvms-5.html#jvms-5.3.1 官网就 2 个

1.3.2.2 扩展类加载器（Extension ClassLoader）

加载 java 平台中扩展功能的一些 jar 包，包括$JAVA_HOME 中 jre/lib/ext/*.jar 或-Djava.ext.dirs（启动指令属性）指定目录下的 jar 包，比如

C:\Program Files\Java\jdk1.8.0_152\jre\lib 下的所有 jar 包的类

具体实现为 sum.misc.Launcher 中的内部类 ExtClassLoader

1.3.2.3 应用程序（系统）类加载器（Application ClassLoader）

加载 classpath 中指定的 jar 包及 Djava.class.path 所指定目录下的类和 jar 包

具体实现为 sum.misc.Launcher 中的内部类 AppClassLoader

1.3.2.4 自定义类加载器（Custom ClassLoader）

前面 3 个是 java 自带的，提供好了的，那么我们也可以自定义！

通过 java.lang.ClassLoader 的子类自定义加载 class，属于应用程序根据自身需要自定义的，ClassLoader，如 tomcat、jboss 都会根据 j2ee 规范自行实现 ClassLoader

-verbose:class

```
    //  System.out.println(ClassB.class.getClassLoader());
    //   System.out.println(Class.forName("org.example.ClassB"));
    //  System.out.println(ClassB.str);
//        Class<?> clazz = Class.forName(ClassB.class.getName());
//        ClassLoader classLoader=clazz.getClassLoader();
//        System.out.println("类加载器是"+classLoader.getClass().getSimp
leName());

//        Class<?> clazz = Class.forName(com.sun.nio.zipfs.ZipPath.clas
s.getName());
//        ClassLoader classLoader=clazz.getClassLoader();
//        System.out.println("类加载器是"+classLoader.getClass().getSimp
leName());
```

咕泡出品 必属精品

```
//        System.out.println(classLoader.getParent().getClass().getSimp
leName());


//        String s1 = new String("he") + new String("llo");
//        String s2 = new String("h") + new String("ello");
//        String s3 = s1.intern();
//        String s4 = s2.intern();
//        System.out.println(s1 == s3);
//        System.out.println(s1 == s4);
```

1.3.2.5 类加载机制

1. 全盘负责：所谓全盘负责，就是当一个类加载器负责加载某个 Class 时，该 Class 所依赖和引用其他 Class 也将由该类加载器负责载入，除非显示使用另外一个类加载器来载入。

2. 双亲委派：所谓的双亲委派，则是先让父类加载器试图加载该 Class，只有在父类加载器无法加载该类时才尝试从自己的类路径中加载该类。通俗的讲，就是某个特定的类加载器在接到加载类的请求时，首先将加载任务委托给父加载器，依次递归，如果父加载器可以完成类加载任务，就成功返回；只有父加载器无法完成此加载任务时，才自己去加载。

3. 缓存机制。缓存机制将会保证所有加载过的 Class 都会被缓存，当程序中需要使用某个 Class 时，类加载器先从缓存区中搜寻该 Class，只有当缓存区中不存在该 Class 对象时，系统才会读取该类对应的二进制数据，并将其转换成 Class 对象，存入缓冲区中。这就是为很么修改了 Class 后，必须重新启动 JVM，程序所做的修改才会生效的原因。

1.3.2.6 双亲委派

类加载的时候，会优先递归给父类加载器去加载，父类加载器没有加载的时候才会往下的加载器去进行加载

目的：为了安全，保证核心类库的安全性，防止被篡改，以及保证类的唯一

ClassNotFoundException（加载时找不到类）；NotClassFoundException（编译时找不到类）

怎么打破双亲委派 重写 classLoad findClass 接口

哈泽出品必属精品

1.3.3 链接(link)

1.3.3.1 验证

验证是连接阶段的第一步，这一阶段的目的是为了确保 Class 文件的字节流中包含的信息符合当前虚拟机的要求，并且不会危害虚拟机自身的安全。

验证文件格式、元数据、字节码等等

文件格式验证：此阶段保证输入的字节流能正确地解析并存储于方法区之内，格式上符合描述一个 Java 类型信息的要求。

（1）是否以魔数 0xCAFEBABE 开头

（2）主、次版本号是否在当前虚拟机处理范围之内 。

（3）常量池的常量中是否有不被支持的常量类型(检查常量 tag 标志)。

（4）指向常量的各种索引值中是否有指向不存在的常量或不符合装型的常量 。

（5）CONSTANT_Utf8_info 型的常量中是否有不符合 UTF8 编码的数据

（6）Class 文件中各个部分及文件本身是否有被删除的或附加的其他信息

元数据验证：保证不存在不符合 Java 语言规范的元数据信息。

（1）这个类是否有父类(除了 java.lang.0bject 之外,所有的类都应当有父类)

（2）这个类的父类是否继承了不允许被继承的类(被 finaI 修饰的类)

（3）如果这个类不是抽象类, 是否实现了其父类或接口之中要求实现的所有方法

（4）类中的字段、方法是否与父类产生了矛盾(例如覆盖了父类的 final 字段, 或者出現不符合规则的方法重载, 例如方法参数都一致, 但返回值类型却不同等)

字节码验证：通过数据流和控制流分析，确定程序语义是合法的、符合逻辑的。例如保证跳转指令不会跳转到方法体以外的字节码指令上。

符号引用验证：在解析阶段中发生，保证可以将符号引用转化为直接引用。

可以考虑使用 `-Xverify:none` 参数来关闭大部分的类验证措施，以缩短虚拟机类加载的时间。

1.3.3.2 准备

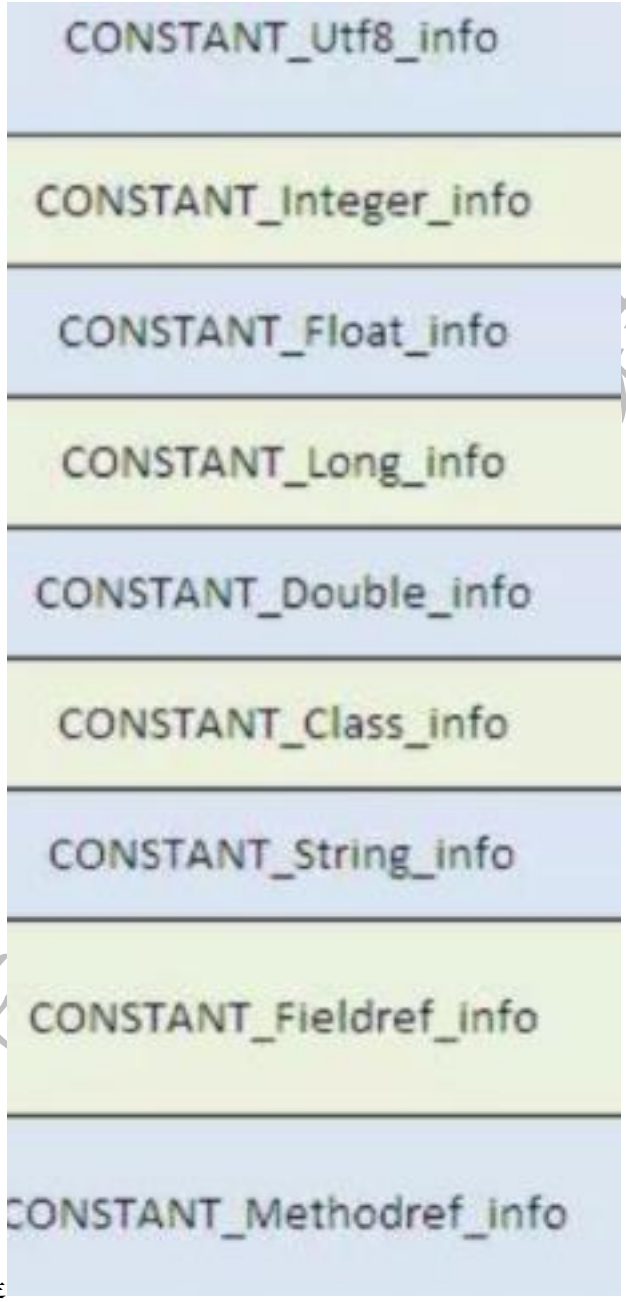
为静态变量分配内存，赋值初始值

咸鱼出品 必属精品

1.3.3.3 解析

将常量池中的符号引用转为直接引用

解析动作主要针对类或接口、字段、类方法、接口方法、方法类型、方法句柄和调用点限定符 7 类符号引用进行。



符号引用：就是

直接引用就是真实地址

1.3.4 初始化（Initialize）

为静态变量赋真正的值。

到初始化阶段，才真正开始执行类中定义的 Java 程序代码，此阶段是执行 `<clinit>()` 方法的过程。

`<init>()` 方法是由编译器按语句在源文件中出现的顺序，依次自动收集类中的所有**类变量**的赋值动作和静态代码块中的语句合并产生的。（不包括构造器中的语句。构造器是初始化对象的，类加载完成后，创建对象时候将调用的 `<init>()` 方法来初始化对象）

静态语句块中只能访问到定义在静态语句块之前的变量，定义在它之后的变量，在前面的静态语句块可以赋值，但是不能访问，如下程序：

```
public class Test {
    static {
        // 给变量赋值可以正常编译通过
        i = 0;
        // 这句编译器会提示"非法向前引用"
        System.out.println(i);
    }

    static int i = 1;
}
```

`<clinit>()` 不需要显式调用父类（接口除外，接口不需要调用父接口的初始化方法，只有使用到父接口中的静态变量时才需要调用）的初始化方法 `<clinit>()`，虚拟机会保证在子类的 `<clinit>()` 方法执行之前，父类的 `<clinit>()` 方法已经执行完毕，也就意味着父类中定义的静态语句块要优先于子类的变量赋值操作。

`<clinit>()` 方法对于类或接口来说并不是必需的，如果一个类中没有静态语句块，也没有对变量的赋值操作，那么编译器可以不为这个类生成 `<clinit>()` 方法。

虚拟机会保证一个类的 `<clinit>()` 方法在多线程环境中被正确地加锁、同步，如果多个线程同时去初始化一个类，那么只会有一个线程去执行这个类的 `<clinit>()` 方法，其他线程都需要阻塞等待，直到活动线程执行 `<clinit>()` 方法完毕。

5 种类必须类初始化情况：

- 在遇到 new、putstatic、getstatic、invokestatic 字节码指令时，如果类尚未初始化，则需要先触发初始化。
- 对类进行反射调用时，如果类还没有初始化，则需要先触发初始化。
- 初始化一个类时，如果其父类还没有初始化，则需要先初始化父类。

必属精品

- 虚拟机启动时，用于需要指定一个包含 main() 方法的主类，虚拟机会先初始化这个主类。
- 当使用 JDK 1.7 的动态语言支持时，如果一个 java.lang.invoke.MethodHandle 实例最后的解析结果为 REF_getStatic、REF_putStatic、REF_invokeStatic 的方法句柄，并且这个方法句柄所对应的类还没初始化，则需要先触发初始化。

所以，我们发现，JVM 的加载肯定是 lazy loading，也就是我们经常讲的懒加载，用到谁加载谁，用不到就不加载

1.3.5 使用

实例化

1.3.6 卸载

Class 被回收要满足以下三个条件:

a) No Instance：该类所有的实例都已经被 GC;

b) No ClassLoader：加载该类的 ClassLoader 实例已经被 GC;

c) No Reference：该类的 java.lang.Class 对象没有被引用。(XXX.class, 静态变量/方法)

1.4 JVM 内存模型

JVM 内存模型就是我刚刚加载的 class 文件数据我要加载到 JVM，那么这些数据我放在 JVM 哪里

1.4.1 方法区(Method Area)

方法区在 JDK8 中的实现是 MeatSpace 元空间 之前就是 Perm Space 永久代

方法区是各个线程共享的内存区域，在虚拟机启动时创建。

Class 文件中除了有类的版本、字段、方法、接口等描述信息外，还有一项信息就是常量池，用于存放编译时期生成的各种字面量和符号引用，这部分内容将在 类加载后进入方法区的运行时常量池中存放。

但是有很多人说字符串常量池已经放到了 heap 里面，但是官网文档还是没有介绍的，我们先过

路漫出品 必属精品

1.4.2 Heap(堆)

Java 堆是 Java 虚拟机所管理内存中最大的一块，在虚拟机启动时创建，被所有线程共享。Java 对象实例以及数组都在堆上分配，被所有线程共享.JAVA 对象实例以及数组都在堆上分配.

1.4.3 Java Virtual Machine Stacks(虚拟机栈)

虚拟机栈是一个线程执行的区域，保存着一个线程中方法的调用状态。换句话说，一个 Java 线程的运行状态，由一个虚拟机栈来保存，所以虚拟机栈肯定是线程私有的，独有的，随着线程的创建而创建。每一个被线程执行的方法，为该栈中的栈帧，即每个方法对应一个栈帧。调用一个方法，就会向栈中压入一个栈帧；一个方法调用完成，就会把该栈帧从栈中弹出

1.4.4 The pc Register（程序计数器）

程序计数器占用的内存空间很小，由于 Java 虚拟机的多线程是通过线程轮流切换，并分配处理器执行时间的方式来实现的，在任意时刻，一个处理器只会执行一条线程中的指令。因此，为了线程切换后能够恢复到正确的执行位置，每条线程需要有一个独立的程序计数器(线程私有)。如果线程正在执行 Java 方法，则计数器记录的是正在执行的虚拟机字节码指令的地址；

1.4.5 Native Method Stacks(本地方法栈)

如果当前线程执行的方法是 Native 类型的，这些方法就会在本地方法栈中执行。

1.5 面试题

详解 JVM 内存模型？有哪些是线程共享，哪些是线程私有

程序计数器：当前线程所执行的字节码的行号指示器，用于记录正在执行的虚拟机字节指令地址，线程私有。

Java 虚拟栈：存放基本数据类型、对象的引用、方法出口等，线程私有。

咕泡出品 必属精品

Native 方法栈：和虚拟栈相似，只不过它服务于 Native 方法，线程私有。

Java 堆：java 内存最大的一块，所有对象实例、数组都存放在 java 堆，GC 回收的地方，线程共享。

方法区：存放已被加载的类信息、常量、静态变量、即时编译器编译后的代码数据等。（即永久代），回收目标主要是常量池的回收和类型的卸载，各线程共享

简单说下类加载器，什么是双亲委派，怎么打破

引导类加载器、拓展类加载器、应用程序（系统）类加载器这个是 java 实现的 3 个加载器，其中引导类由 c++语言实现，负责 jre/lib/rt.jar，扩展类跟应用程序由 java 语言实现，继承自抽象类 ClassLoader

展类加载器（Extension ClassLoader）：负责加载<JAVA_HOME>\lib\ext 目录或 java.ext.dirs 系统变量指定的路径中的所有类库。
应用程序类加载器（Application ClassLoader）。负责加载用户类路径（classpath）上的指定类库，我们可以直接使用这个类加载器。一般情况，如果我们没有自定义类加载器默认就是用这个加载器。

双亲委派就是每个类加载器收到类加载请求的时候，首先不会尝试加载这个类，而是给到父类加载器去加载，当父类加载器没有然后递归给子类加载！！

目的是为了保证类的唯一性，以及系统类的安全，不被篡改

打破双亲委派机制则继承 ClassLoader 类，重写 loadClass 和 findClass 方法。

描述下 JAVA class 类的生命周期

1.6 补充

1.6.1 invoke

具体来说,Java 字节码中与调用相关的指令共有五种:

1.invokestatic:用来调用静态方法，即使用 static 关键字修饰的方法。 它要调用的方法在编译期间确定，运行期不会修改，属于静态绑定。它也是所有方法调用指令里面最快的。

例如：

```
String.valueOf(123)
```

咕泡出品必屬精品

2.invokespecial:它是特殊的方法调用，包括实例构造方法、私有方法（private 修饰的方法）和父类方法（即 super 关键字调用的方法）。很明显明显，这些特殊的方法可以直接确定实际执行的方法的实现，与 invokestatic 一样，也属于静态绑定。

3.invokevirtual:用来调用 public、protected、package 访问级别的方法。方法要根据对象类型不同动态选择不同的类，在编译期不确定，属于动态绑定。

例如：

```
public class Car {
    public void printCarName()  {
        System.out.println("Car name from parent");
    }
}
public class Audi extends Car {
    @Override
    public void printCarName() {
        System.out.println("Car name is Audi");
    }
}
public class BMW extends Car {
    @Override
    public void printCarName() {
        System.out.println("Car name is BMW");
    }
}
public class InvokeVirtualTest {
    private static Car audiCar = new Audi ();
    private static Car bmwCar= new BMW ();
    public static void main(String[] args) {
        audiCar.printCarName();
        bmwCar.printCarName();
    }
}
```

```
0: getstatic     #2                  // Field audiCar :LCar;
3: invokevirtual #3                  // Method Car.printCarName:()V
6: getstatic     #4                  // Field bmwCar:LCar;
9: invokevirtual #3                  // Method Car.printCarName:()V
```

invokevirtual 会根据对象的实际类型进行分派（虚方法分派），在编译期间不能确定最终会调用子类还是父类的方法。

4.invokeinterface:用于调用接口方法，在运行时再确定一个实现此接口的对象，也属于动态绑定。

咕泡出品 必属精品

5.invokedynamic:**出现于 JDK8 中的 lambda 表达式中，是一种调用方法的新方式，它用来告诉 JVM 可以延迟确认最终要调用的哪个方法。一开始 invokedynamic 并不知道要调用什么目标方法。第一次调用时引导方法（Bootstrap Method）会被调用，由这个引导方法决定哪个目标方法进行调用。

在 Java 中，普通的方法调用的类名、方法名、入参类型、返回类型都是确定的：

```
audiCar.printCarName("audi car");
```

但 Java 还可以通过方法句柄 MethodHandle 来调用方法：

```
public class Foo {
    public void printCarName(String s) {
        System.out.println(s);
    }
    public static void main(String[] args) throws Throwable {
        Foo foo = new Foo();
        MethodType methodType = MethodType.methodType(void.class, Strin
g.class);
        MethodHandle methodHandle = MethodHandles.lookup().findVirtual
(Foo.class, "printCarName", methodType);
        methodHandle.invokeExact(foo, "audi car");
    }
}
```

运行输出

```
audi car
```

咕泡出品 必属精品